Bank Account Management System

Java Program using Inheritance and Method Overriding

1. Question

Design and implement a Bank Account Management System in Java using inheritance and method overriding. Create a base class **Account** with account number, account holder name and balance. Implement deposit, withdrawal, transfer and display operations. Create derived classes **SavingsAccount** and **CurrentAccount** with their own withdrawal and interest/charge operations.

2. Steps of the Program

1. Create a base class Account.
2. Declare account number and account holder as private data members.
3. Declare balance as a data member.
4. Create a parameterized constructor to initialize account details.
5. Create getter methods for account number, account holder and balance.
6. Create a deposit() method to add money to the account.
7. Create a withdraw() method that can be overridden by child classes.
8. Create a calinterest() method that can be overridden.
9. Create a transfer() method to transfer money from one account to another.
10. Create a show() method to display account details.
11. Create SavingsAccount extending Account.
12. Add interestrate to SavingsAccount.
13. Override withdraw() and calinterest() in SavingsAccount.
14. Create CurrentAccount extending Account.
15. Add overdraftlimit to CurrentAccount.
16. Override withdraw() and calinterest() in CurrentAccount.
17. Create objects for SavingsAccount and CurrentAccount.
18. Perform deposit, withdrawal, interest/charge and transfer operations.
19. Display the final account details.

3. Algorithm

Step 1: Start.

Step 2: Create an Account class with account number, account holder and balance.

Step 3: Initialize account details using a constructor.

Step 4: Implement deposit() to add the given amount to the balance.

Step 5: Implement withdraw() and calinterest() so that they can be overridden.

Step 6: Implement transfer() to withdraw money from one account and deposit it into another account.

Step 7: Create SavingsAccount as a subclass of Account.

Step 8: Initialize the savings account with an interest rate.

Step 9: Override withdraw() to check sufficient balance.

Step 10: Override calinterest() to calculate and add interest to the balance.

Step 11: Create CurrentAccount as a subclass of Account.

Step 12: Initialize the current account with an overdraft limit.

Step 13: Override withdraw() to allow withdrawal up to the overdraft limit.

Step 14: Override calinterest() to add the maintenance charge.

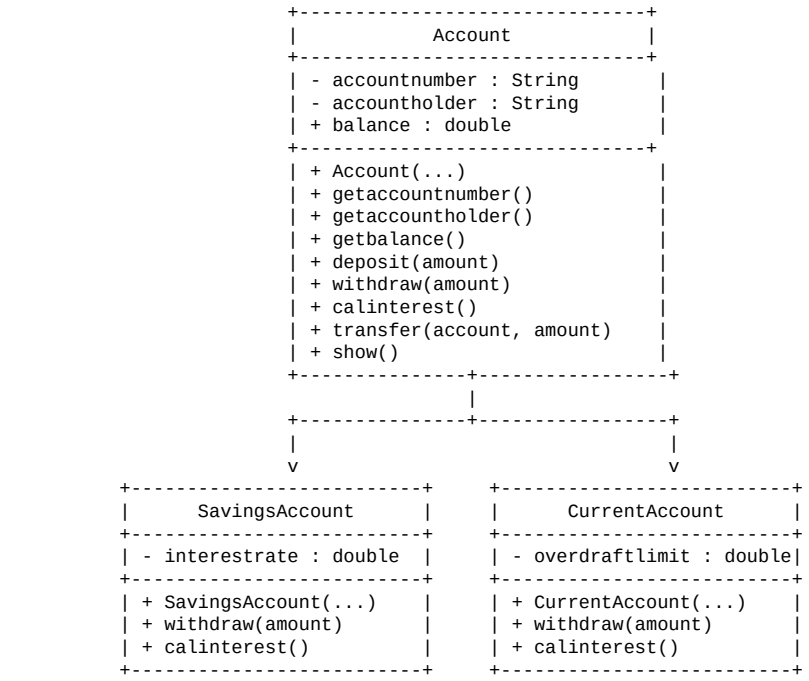
Step 15: Create objects for savings and current accounts.

Step 16: Perform deposit, withdrawal, interest/charge and transfer operations.

Step 17: Display the final account details.

Step 18: Stop.

4. UML Diagram



UML Relationship:
SavingsAccount ---------|> Account
CurrentAccount ---------|> Account
(The arrow ---------|> represents inheritance.)

5. Java Program (Source Code)

```
package packagecodejava;

class Account {
    private String accountnumber;
    private String accountholder;
    public double balance;

    public Account(String accountnumber, String accountholder, double balance) {
        this.accountnumber = accountnumber;
        this.accountholder = accountholder;
        this.balance = balance;
    }

    public String getaccountnumber() {
        return accountnumber;
    }

    public String getaccountholder() {
        return accountholder;
    }

    public double getbalance() {
        return balance;
    }

    void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("deposit amount is greater than zero");
            return;
        }
        balance += amount;
        System.out.println(" the deposited amount is " + amount);
    }

    void withdraw(double amount) {
    }

    void calinterest() {
    }

    void transfer(Account fixedAccount, double amount) {
        System.out.println("transfer of" + amount + "from" + accountnumber
                + "to" + fixedAccount.getaccountnumber() + "---");
        this.withdraw(amount);
        fixedAccount.deposit(amount);
    }

    void show() {
        System.out.println("ACCOUNT NUMBER IS " + accountnumber);
        System.out.println("ACCOUNT HOLDER IS " + accountholder);
        System.out.println("BALANCE IS " + balance);
    }
}

class SavingsAccount extends Account {
    private double interestrate;

    public SavingsAccount(String accountnumber, String accountholder, double balance) {
        super(accountnumber, accountholder, balance);
        this.interestrate = interestrate;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("with draw amount is not possible ");
        } else if (amount >= balance) {
            System.out.println("it has insufficient balance");
        } else {
```

```
            balance -= amount;
            System.out.println("withdraw is " + amount);
        }
    }

    @Override
    void calinterest() {
        double interest = balance * interestrate;
        balance += interest;
        System.out.println("interest is credited" + interest);
    }
}

class CurrentAccount extends Account {
    private double overdraftlimit;

    public CurrentAccount(String accountnumber, String accountholder, double balance) {
        super(accountnumber, accountholder, balance);
        this.overdraftlimit = overdraftlimit;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= 0)
            System.out.println("with draw amount is not possible ");
        else if (amount > balance + overdraftlimit) {
            System.out.println("it has insufficient balance and the limited is done");
        } else {
            balance -= amount;
            System.out.println("withdraw is " + amount);
        }
    }

    @Override
    void calinterest() {
        double charge = 50.0;
        balance += charge;
        System.out.println("maintainence charge is credited" + charge);
    }
}

public class Week01 {
    public class LabProgram1 {
        public static void main(String[] args) {
            CurrentAccount ca = new CurrentAccount("UB 24311", "kumar", 4500);
            SavingsAccount sa = new SavingsAccount("sb 24533", "heamnth", 5000);

            System.out.println(" initial account details");
            ca.show();
            System.out.println("");
            sa.show();

            System.out.println("transaction details");
            ca.deposit(2000);
            ca.withdraw(2500);
            ca.calinterest();

            sa.withdraw(3000);
            sa.calinterest();

            sa.transfer(ca, 1000);

            System.out.println("final account details");
            ca.show();
            System.out.println("");
            sa.show();
        }
    }
}
```

```
================================================
          LIBRARY MANAGEMENT SYSTEM
================================================


--- PART A & B: BOOK DETAILS ---

Book 1 - Default Constructor + Setters
Book ID   : 101
Book Name : Java Programming
Author    : James Gosling
Price     : Rs. 450.0

Book 2 - Parameterized Constructor
Book ID   : 102
Book Name : Object Oriented Programming
Author    : Robert Lafore
Price     : Rs. 550.0

Accessing Book 2 using Getter Methods:
Book ID   : 102
Book Name : Object Oriented Programming
Author    : Robert Lafore
Price     : Rs. 550.0


--- PART C: INHERITANCE ---

Student Details:
Name : Rahul
Age  : 20
Student ID : 501
Course     : Computer Science

Faculty Details:
Name : Dr. Priya
Age  : 40
Faculty ID  : 301
Department  : Computer Science


--- PART D: METHOD OVERLOADING ---
Area of Circle = 78.53981633974483
Area of Rectangle = 50.0
Area of Square = 36.0
Area of Triangle = 16.0


--- METHOD OVERRIDING ---
This is a Car.
This is a Bike.


--- PART E: ABSTRACTION ---
Drawing a Circle.
Drawing a Rectangle.


--- INTERFACE ---
Printing the Library Report.

================================================
       PROGRAM EXECUTED SUCCESSFULLY
```